Topic 7: API Integration Patterns & Pagination

API Authentication Methods Requirement File

1. Document Control

Field	Value
Document Title	Topic 7: API Authentication Methods Requirement File
Version	1.0
Status	Approved
Last Updated	2026-06-20
Prepared By	Lead Automation Architect (Antigravity AI)
Target Audience	Automation Builders, System Integrators, QA Engineers

2. Learning Concepts

2.1 API Authentication Overview

Application Programming Interfaces (APIs) require authentication to verify the identity of the client making the request and to ensure they have permission to access the requested resource. Authenticating requests is the first step in establishing secure communication channels between n8n and external business systems (such as CRMs, billing systems, or social platforms).

2.2 API Key Authentication

An API Key is a unique, secret string generated by a service to identify a project or user. It functions like a long-term password.

- **Mechanism:** API Keys are typically sent in one of two ways:

  1. **HTTP Headers:** Sent in a custom header (e.g., `X-API-KEY: your_api_key_here`). This is the preferred method as headers are generally encrypted in transit (via HTTPS).

  2. **Query Parameters:** Appended to the URL (e.g., `https://api.example.com/v1/data?api_key=your_api_key_here`). This is less secure because URLs are often logged by web servers, proxies, and browser histories.

- **Security Consideration:** API Keys do not expire automatically and must be rotated manually if compromised.

2.3 Bearer Token Authentication

Bearer Token authentication (often implemented using JSON Web Tokens - JWT) is a security mechanism where the client presents a token in the authorization header. The server trusts the “bearer” of the token without requiring additional credentials.

- **Mechanism:** The token is sent in the standard `Authorization` header prefixed with the word `Bearer`:
  ```http
  Authorization: Bearer your_jwt_access_token_here
  ```
- **Security Consideration:** Bearer tokens are typically short-lived (e.g., expiring in 15 minutes to a few hours) and are issued after a separate authentication process (such as a username/password login or client credential exchange).

2.4 OAuth2 Authentication

OAuth2 is an open-standard authorization framework that allows third-party applications to obtain limited access to a service on behalf of a user without sharing the user's password.

- **Key Concepts:**
  - **Authorization Code Flow:** Used when a user needs to log in interactively. The user is redirected to the service provider, logs in, grants permission, and is redirected back to n8n with an Authorization Code. n8n then exchanges this code for an Access Token and a Refresh Token.
  - **Client Credentials Flow:** Used for machine-to-machine communication where no user interaction is required. The client exchanges its Client ID and Client Secret directly for an access token.
  - **Access and Refresh Tokens:** The Access Token is short-lived. When it expires, n8n automatically uses the long-lived Refresh Token to request a new access token without interrupting the workflow or requiring manual re-authentication.
  - **Redirect URI:** The callback URL configured on the service provider's developer console (e.g., `https://your-n8n-instance.com/rest/oauth2-credential/callback`).

2.5 Best Practices for Credential Security in n8n

To maintain data integrity and security, developers must never hardcode API keys or secret tokens directly inside HTTP Request node parameters.

- **n8n Credential Store:** Credentials should be created in n8n's centralized, encrypted credential store. This separates sensitive credentials from the workflow definition, ensuring that secrets are not exposed when workflows are exported as JSON.
- **Credential Sharing:** Configure sharing permissions inside n8n to grant workspace members access to use credentials without allowing them to view the raw secret keys.

---

3. Hands-on Concepts with Syntax & Examples

3.1 API Key Header Configuration

To configure an API Key manually or via n8n's header auth:

- **Header Name:** `X-API-Key`
- **Header Value:** `{{ $credentials.apiKey }}` (or standard credential binding)

3.2 Bearer Token Configuration

In the n8n HTTP Request node, when selecting the Bearer Token authentication type:

- **Token:** Enter the bearer secret token.

- **HTTP Output:** n8n automatically appends the following header to the outgoing request:

  ```http
  Authorization: Bearer
  ```

3.3 OAuth2 Credential Binding in n8n

To connect an OAuth2 API (e.g., Google APIs or Salesforce), the n8n credential configuration requires:

- **Grant Type:** Authorization Code
- **Authorization URL:** `https://provider.com/oauth/authorize`
- **Access Token URL:** `https://provider.com/oauth/token`
- **Client ID:** Generated in the provider's developer console.
- **Client Secret:** Generated in the provider's developer console.
- **Scope:** Space or comma-separated permissions list (e.g., `read:contacts write:contacts`).
- **OAuth Redirect URL:** Copy the URL provided by n8n (e.g., `https://n8n.domain.com/rest/oauth2-credential/callback`) and paste it into the provider's redirect whitelist.

---

4. Business Requirements Document (BRD)

4.1 Business Objectives

To establish secure, standardized, and auditable connection patterns between the company's n8n automation workspace and external cloud services, minimizing the risk of credential exposure and automating token refresh cycles.

4.2 Business Requirements Table

ID	Business Requirement	Business Rationale	Priority
BRD-001	Secrets and authorization tokens shall never be hardcoded into workflow node parameters.	Prevents exposure of sensitive API keys in exported JSON logs or git repositories.	High
BRD-002	Connections to external services requiring user authorization shall utilize OAuth2.	Avoids sharing passwords and ensures that user permissions can be revoked remotely.	High
BRD-003	Authorization tokens must be automatically refreshed by the automation engine.	Prevents integration pipelines from failing due to credential expiration.	High
BRD-004	The system shall support custom headers for legacy API Key validation.	Ensures compatibility with internal proprietary microservices.	Medium

---

5. Product Requirements Document (PRD)

5.1 Product Capabilities

ID	Product Requirement	User Value	Priority
PRD-001	Encrypted Credential Vault	Encrypts and isolates keys from workflow logic.	High
PRD-002	Automated OAuth2 Token Renewer	Handles refresh token rotation transparently without admin intervention.	High
PRD-003	Standard Auth Template Selector	Provides a dropdown to switch between API Key, Bearer Token, and OAuth2.	High
PRD-004	Error-Handling Callback Listener	Captures auth failures (401 Unauthorized) and throws clean alerts.	Medium

6. Software Requirements Specification (SRS) / Technical Requirements

6.1 Functional Requirements

- **FR-001:** The system shall support three distinct credential types: API Key (Header/Query), Bearer Token, and OAuth2 (Authorization Code).
- **FR-002:** When using API Key authentication, the HTTP node shall append the key to either the HTTP Headers or Query Parameters as specified by the user.
- **FR-003:** When using Bearer Token authentication, the HTTP node shall automatically construct and inject the `Authorization: Bearer` header.
- **FR-004:** When using OAuth2 authentication, the system shall manage the initial authorization code handoff and save the returned access and refresh tokens.
- **FR-005:** Prior to executing an HTTP request with an expired OAuth2 access token, the system shall make a background POST request to the Access Token URL using the refresh token to update the access credentials.
- **FR-006:** The system shall encrypt all saved credentials at rest using a unique workspace-level encryption key.

6.2 Non-Functional Requirements

- **NFR-001 (Security):** Under no circumstances shall credentials configured via n8n's Credential Store be serialized or exported when the user performs a "Download Workflow" action.
- **NFR-002 (Reliability):** The token refresh cycle must execute in under 1 second, preventing timeouts in the primary request thread.

7. User Stories & Acceptance Criteria

US-001: API Key Header Injection

- **User Story:** As a Workflow Builder, I want to authenticate against our internal billing API using an API key stored in the credentials vault, so that I do not leak the key when sharing my workflow.
- **Acceptance Criteria:**
  - Given an API Key credential named "Billing API Key" is configured in n8n,

- When an HTTP Request node executes a GET request using this credential,
- Then the request must include the header `X-API-Key` with the encrypted value, and the exported workflow JSON must display `"authentication": "predefinedCredentialType"`.

US-002: Automated OAuth2 Refresh Cycle

- **User Story:** As an Operations Manager, I want our HubSpot integration to run continuously without requiring me to re-authorize the OAuth connection every hour.
- **Acceptance Criteria:**
  - Given an OAuth2 connection with an access token that expired 5 minutes ago,
  - When a workflow runs an HTTP Request node connected to that credential,
  - Then the engine must request a new access token from the auth provider using the refresh token, save the new token, and execute the original HTTP call successfully.

---

8. Test Cases

ID	Test Case	Preconditions	Steps	Expected Result	Priority	Linked Requirement
TC-001	API Key in HTTP Header	Mock API server requires `X-API-Key: secret123`.	Configure Header Auth in n8n. Execute GET request.	Request succeeds with status 200. Inspect server logs to verify header.	High	FR-001, FR-002
TC-002	Bearer Token Auth	Mock API server requires Bearer JWT token.	Configure Bearer Token credential. Execute request.	Header contains `Authorization: Bearer`. Request succeeds.	High	FR-001, FR-003
TC-003	Expired OAuth2 Token Refresh	Access token is set to expired; refresh token is valid.	Trigger a GET request via the OAuth2 authorized node.	Node pauses, requests new access token, receives it, updates storage, and completes call with status 200.	High	FR-004, FR-005
TC-004	Failed Token Refresh Alert	Both access and refresh tokens are revoked or invalid.	Trigger the HTTP Request node.	Request fails. Workflow outputs a 401 Authentication Error with clear error logs.	High	FR-005
TC-005	Security Export Validation	Workflow is fully configured with credentials.	Export the workflow JSON using “Download”. Search JSON for secret values.	The exported JSON contains no plain-text secret keys or tokens.	High	NFR-001

---

9. Assumptions, Dependencies, and Risks

9.1 Assumptions

- **ASM-001:** The external API providers follow RFC compliance for OAuth2 and Bearer Token structures.

- **ASM-002:** The n8n server has outbound HTTPS access to contact the provider's token validation and refresh endpoints.

9.2 Dependencies

- **DEP-001:** n8n instance version 1.0.0+ with encrypted database storage enabled.

9.3 Risks & Mitigation

- **RSK-001 (Refresh Token Expiration):** Some OAuth2 providers expire refresh tokens after inactivity (e.g. 14 days of no calls) or rotate them, which can break workflows.
  - **Mitigation:** Implement a health check workflow that triggers the endpoint once a day to ensure the token remains rotated and active.

---

10. Glossary

Term	Definition
Authorization Code	A temporary code passed from the auth server back to the client, which is exchanged for an access token.
Access Token	A short-lived credential used directly to authenticate requests to protected APIs.
Refresh Token	A long-lived credential used exclusively to request new access tokens when they expire.
Encrypted Vault	Centralized credential repository within n8n where credentials are encrypted before database insertion.
JWT	JSON Web Token, an open standard that defines a compact and self-contained way for securely transmitting information.

---

11. Traceability Matrix

Source Item	BRD	PRD	FR	User Story	Test Case
BRD-001	BRD-001	PRD-001	FR-006	US-001	TC-005
BRD-002	BRD-002	PRD-003	FR-001, FR-004	n/a	TC-003
BRD-003	BRD-003	PRD-002	FR-005	US-002	TC-003, TC-004
BRD-004	BRD-004	PRD-003	FR-002, FR-003	US-001	TC-001, TC-002